DAYKIN

Docker Compose to Kubernetes Migration Discovery & Plan

Working document • Prepared 20 October 2026

Purpose. Record the current Daykin deployment as discovered on the VPS, define the intended Kubernetes architecture, and establish a safe migration sequence. This is a planning document—not an instruction to deploy immediately.

Current state	Target state
Docker containers and Compose files on one Ubuntu VPS.	Kubernetes-managed workloads, stable Services, planned persistent storage, and tested routing.
Nginx Proxy Manager already uses host ports 80, 81, and 443.	Preserve existing proxy and other hosted applications; route traffic only after a deliberate cutover.

Migration principle: inventory → backup → isolated test → validation → planned cutover → cleanup only after stability.

1. Executive summary

Daykin currently runs as Docker containers on an Ubuntu 24.04.2 LTS VPS with 3 vCPUs, approximately 7.8 GiB RAM, and a 145 GB root filesystem (about 94 GB available at the time of inspection). Backend services are managed through Docker Compose, while the frontend has a separate Compose file. Nginx Proxy Manager already occupies host ports 80, 81, and 443. Any Kubernetes plan must preserve those bindings and the other applications hosted on the VPS.

The migration should be staged and reversible. Back up and verify data first, build Kubernetes manifests separately, test in an isolated namespace or environment, and only change production routing after storage, health checks, application behavior, and rollback have been validated.

2. Current environment (observed)

Item	Observed value
Host OS	Ubuntu 24.04.2 LTS; x86-64; KVM virtual machine
Compute	3 vCPUs; approximately 7.8 GiB RAM
Disk	Root filesystem 145 GB total, 51 GB used, approximately 94 GB available at inspection
Container platform	Docker and Docker Compose
Public proxy	Nginx Proxy Manager container; host ports 80, 81, and 443
Domain	ebiseera.com
Frontend API base found in built bundle	https://ebiseera.com/api
Daykin Nginx published ports	VPS 8085 → container 80; VPS 9443 → container 443, according to backend Compose file
Frontend published port	VPS 3006 → frontend container port 80

3. Current Daykin architecture

Observed components and request flow (the exact Nginx Proxy Manager upstream rule still needs verification):

- **Frontend:** image `kennedymutebi/daykin-frontend:latest`, container `daykin_frontend`, host port 3006 mapped to container port 80.
- **Daykin Nginx:** image `nginx:alpine`, host port 8085 mapped to container port 80 and host port 9443 mapped to container port 443. Configuration file: `/root/daykin/backend/nginx.conf`.
- **API routing:** Nginx forwards requests under `/api/` to `http://daykin_backend:8000`.
- **Media and static files:** Nginx serves `/media/` from `/app/media/` and `/static/` from `/app/staticfiles/`.
- **Django backend:** image `kennedymutebi/daykin-backend:18026a3fd3278771ffe3f5484c3984110b0cfa05`, container `daykin_backend_web`, internal port 8000.
- **Background jobs:** Celery worker and Celery Beat use the backend image. Keep Beat at one active replica initially to avoid duplicate scheduled jobs.
- **Data services:** MySQL 8.0 container `daykin_db` and Redis 7 container `daykin_redis`.

Frontend API finding: searching the built frontend bundle found `https://ebiseera.com/api`. The same output also contained localhost URLs; those may be code defaults or unrelated paths and need source-level verification.

4. Current storage and networking

Docker volumes confirmed

Docker volume	Container path	Purpose / migration note
backend_daykin_db_data	/var/lib/mysql	MySQL database files; critical persistent data.
backend_daykin_media	/app/media	Uploaded media; preserve and verify backup.
backend_daykin_static	/app/staticfiles	Collected static files; may be rebuildable, but verify the collection process.

Docker volume	Container path	Purpose / migration note
backend_daykin_redis_data	/data	Redis persistence volume; confirm whether Redis holds business-critical data or only cache/broker state.

Docker networks observed

The backend and Daykin Nginx share Docker network `backend_default`. At inspection, the backend IP was 172.22.0.5 and Nginx was 172.22.0.4. These addresses are temporary and must not be hard-coded into Kubernetes manifests. The frontend is attached to Docker's `bridge` network (observed IP 172.17.0.5), not `backend_default`. The built frontend appears to call the public domain for API requests.

5. Proposed Kubernetes target architecture

This is a proposed design. Resource types, storage, routing, and replica counts must be validated against the selected Kubernetes distribution and available storage before implementation.

Component	Proposed Kubernetes resource	Design notes
Frontend	Deployment + ClusterIP Service	Serve frontend image in the cluster and expose it through the chosen proxy/routing arrangement.
Backend API	Deployment + ClusterIP Service	Django pods listen on port 8000; connect through a stable Service name, never pod IPs.
Celery worker	Deployment	Run background jobs using the same pinned backend image and approved configuration.
Celery Beat	Deployment, one replica initially	Use one scheduler unless distributed scheduling is explicitly designed.
MySQL	Managed DB if available; otherwise StatefulSet + PVC	Back up and test restore. Migrate data deliberately; do not assume the Docker volume can be mounted directly.
Redis	Deployment or StatefulSet + PVC if required	Choose persistence based on its role and acceptable data-loss window.
Uploaded media	PVC or shared object storage	All backend replicas and the media-serving component must access the same files if the app scales.
Static files	Image-baked assets or dedicated static serving	Avoid unnecessary mutable shared storage if assets can be collected during image build/deployment.
External routing	Keep Nginx Proxy Manager initially	Route to a tested reachable upstream. Do not unexpectedly take over host ports 80/443.
Configuration and secrets	ConfigMaps + Kubernetes Secrets or external secret manager	Keep secrets out of Git, images, and plain manifests.

6. Important design principles

- Kubernetes Services provide stable internal names; individual pod IP addresses can change.
- Docker named volumes do not automatically become Kubernetes PersistentVolumeClaims (PVCs). Data migration and restore must be planned.
- Add readiness and liveness probes only after identifying reliable health endpoints and startup behavior.
- Compose `depends_on` behavior does not translate directly to Kubernetes. Applications should tolerate database/Redis delays and retry safely.
- Pin tested image tags or digests for repeatable releases; avoid relying on `latest` for a production cutover.
- Keep production and other hosted applications untouched while building and testing the Kubernetes version.
- Resource requests and limits must account for the VPS's 3 vCPUs and the memory needed by Kubernetes system components plus all workloads.
- Back up MySQL and uploaded media, and test a restore before any production data migration.

7. Migration phases and exit criteria

Phase 0 — Discovery (current phase)

- Record container images, Compose services, ports, volumes, networks, proxy rules, and environment variable names (never secret values).
- Confirm which frontend/backend containers are authoritative and actively serving users.
- Verify API endpoints, TLS termination, allowed hosts/CORS/CSRF settings, and background-job behavior.
- **Exit:** inventory reviewed, unknowns recorded, and no production changes made.

Phase 1 — Backups and recovery

- Create a consistent MySQL backup and archive uploaded media; preserve existing Docker volumes.
- Back up Compose files, Nginx configuration, proxy settings, and image references securely.
- Verify backup integrity and rehearse restoration in an isolated environment.
- **Exit:** documented restore steps succeed.

Phase 2 — Kubernetes foundation in isolation

- Choose the cluster approach and storage class; confirm no port conflicts with Nginx Proxy Manager or other apps.
- Create a namespace, configuration, secrets, services, deployments, probes, and PVCs in a test scope.
- Do not point test pods at the production database without an explicitly approved test plan.
- **Exit:** pods become Ready and internal service discovery works.

Phase 3 — Application and data validation

- Deploy frontend, backend, worker, Beat, and test data services using pinned images.
- Restore a database copy and media copy into test storage.
- Test login, API calls, uploads, static assets, scheduled jobs, integrations, logs, and restart recovery.
- **Exit:** functional checks pass and backups remain restorable.

Phase 4 — Traffic cutover

- Write a cutover and rollback checklist; schedule a maintenance window if needed.
- Route Nginx Proxy Manager to the tested Kubernetes entry point without unexpected host-port changes.
- Verify HTTPS, DNS, API behavior, uploads, database writes, and logs.
- **Exit:** application owner confirms success; old deployment and backups remain available during a rollback window.

Phase 5 — Cleanup (only after stability)

- Remove old resources only after validation, backup verification, and an agreed rollback window.
- Never delete Docker volumes until their contents are confirmed migrated and recoverable.

8. Risks and open questions

- Which Nginx Proxy Manager Proxy Host points to Daykin, and what upstream hostname and port are configured?
- Is `https://ebiseera.com/api` the correct API URL for every environment? What are the localhost URL references in the compiled bundle?
- Which frontend container is authoritative? Another container named `daykin` is also present on host port 8083; determine whether it is legacy or active before cleanup.
- Is MySQL exposed on a host port, or reachable only over the internal Docker network? Verify Docker port metadata and firewall rules.
- What Redis data is stored, and what loss is acceptable? Is Redis used as Celery broker/cache only?
- A MySQL credential was exposed during troubleshooting. Treat it as compromised and plan a safe rotation. Changing `MYSQL_ROOT_PASSWORD` in Compose alone does not rotate an existing MySQL account.
- Does the VPS have an appropriate Kubernetes storage class and enough resource headroom? A previous K3s attempt was removed after it interfered with ports/apps, so an isolation and port plan is essential.

- How are static files collected, and does the backend require shared media access when scaled?
- What health endpoints exist, and can the application retry database/Redis connections during startup?

9. Current safety rules

- No Kubernetes installation or production migration has been performed as part of this discovery.
- Do not stop, remove, or recreate running containers during inventory.
- Do not prune Docker volumes or delete database/media volumes.
- Do not paste .env files, passwords, tokens, private keys, or secret values into chat or Git.
- Before a future cutover, make and verify backups and agree on rollback steps.

10. Next discovery steps

- Inspect Nginx Proxy Manager's Proxy Host for ebiseera.com and record its upstream host/port without modifying it.
- Inspect frontend API configuration in source, if available; confirm the public API URL and explain localhost references.
- Capture safe metadata for image tags/digests, ports, health status, volume mounts, and environment variable names only.
- Review backend Compose behavior for Redis and health checks without exposing environment values.
- Decide whether Kubernetes will be tested on this VPS or on a separate host before choosing ingress and storage.

Document status: Discovery and planning draft. This document reflects information observed so far and marks unverified details as open questions. It does not authorize changes to the production VPS.